

Práctica de Laboratorio #5 Puertos Paralelos en C

Objetivo

Que el estudiante aprenda el procedimiento necesario para controlar los puertos paralelos de una tarjeta de entrenamiento de los microcontroladores STM32F3 y STM32G4 utilizando el IDE VS Code y Keil MDK6.

Teoría

Las familias de microcontroladores STM32F303 y STM32G431 cuentan con hasta 7 puertos paralelos bidireccionales de propósito general: Los puertos A, B, C, D, E, F y G de 16 pines cada uno (depende de la versión del microcontrolador).

Cada puerto tiene dos registros de datos asociados (**GPIOx_ODR** y **GPIOx_IDR**), para escritura y lectura de datos. Si se escribe al **Output Data Register (ODR)**, se escribe a los bits configurados como salidas, y estos estados aparecerán en los pines asociados. Los bits configurados como entradas pueden ser leídos por medio del **Input Data Register (IDR)**, y estos tienen el estado lógico de los pines asociados al mismo.

Cada puerto tiene también un registro de configuración (**Mode Register GPIOx_MODER**) utilizado para configurar la dirección de los datos y en la que también se puede configurar la funcionalidad alternativa de cada pin. Este se utiliza para configurar los bits del puerto de forma individual, ya sea como entrada, como salida, como su función alternativa o como un pin de entrada o salida analógica.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y+1:2y **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

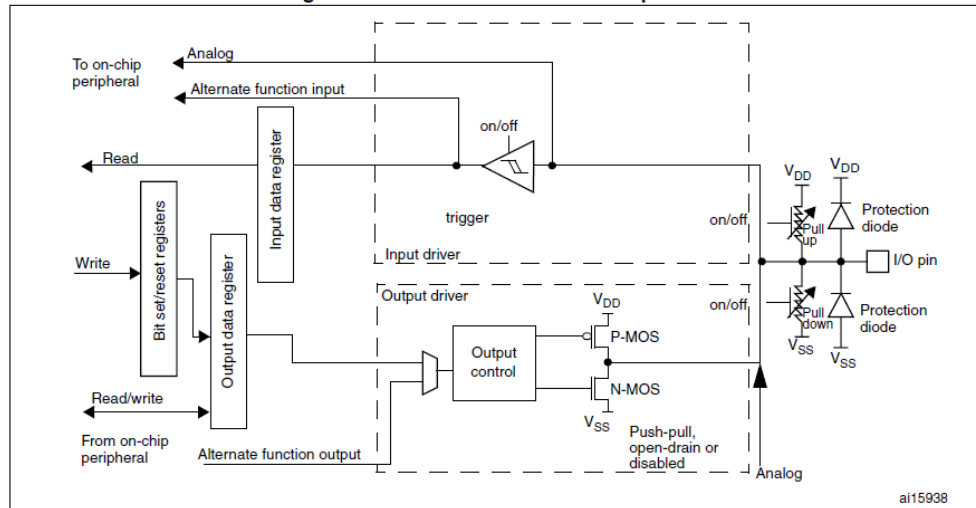
11: Analog mode

Note que cada pin necesita de dos bits de configuración, por lo tanto, si usted desea configurar el pin 10, es necesario modificar los bits 20 y 21 del **Mode Register**.

Cada pin de cada puerto, como ya fue mencionado, tiene también una función alternativa que puede ser configurada en un registro llamado **Alternate Function Register**, en el cual cada pin necesita de cuatro bits de configuración para seleccionar la función alternativa asociada al mismo, por lo tanto, este registro se divide en dos registros de 32 bits (**GPIOx_AFHR** y **GPIOx_AFLR**). Para esta práctica no es necesario realizar configuraciones en este registro.

Tómese unos minutos para revisar los siguientes diagramas del hardware de los pines del microcontrolador. ¿Le parece complicado? ¿Entiende su funcionamiento?

Figure 40. Basic structure of an I/O port bit



Como podrá notar en los diagramas, es posible hacer set y reset de forma individual a cada uno de los pines configurados como salidas utilizando el **Port Bit Set/Reset Register (GPIOx_BSRR)**.

Bits 31:16 **BRy**: Port x reset bit y (y = 0..15)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x set bit y (y = 0..15)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

Además, es posible añadir pull-up y/o pull-down resistors, siempre y cuando el pin fue configurado como entrada digital y este resistor sea requerido. Esta funcionalidad puede ser configurada en el **Port pull-up/pull-down register (GPIOx_PUPDR)**.

Bits 2y+1:2y **PUPDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

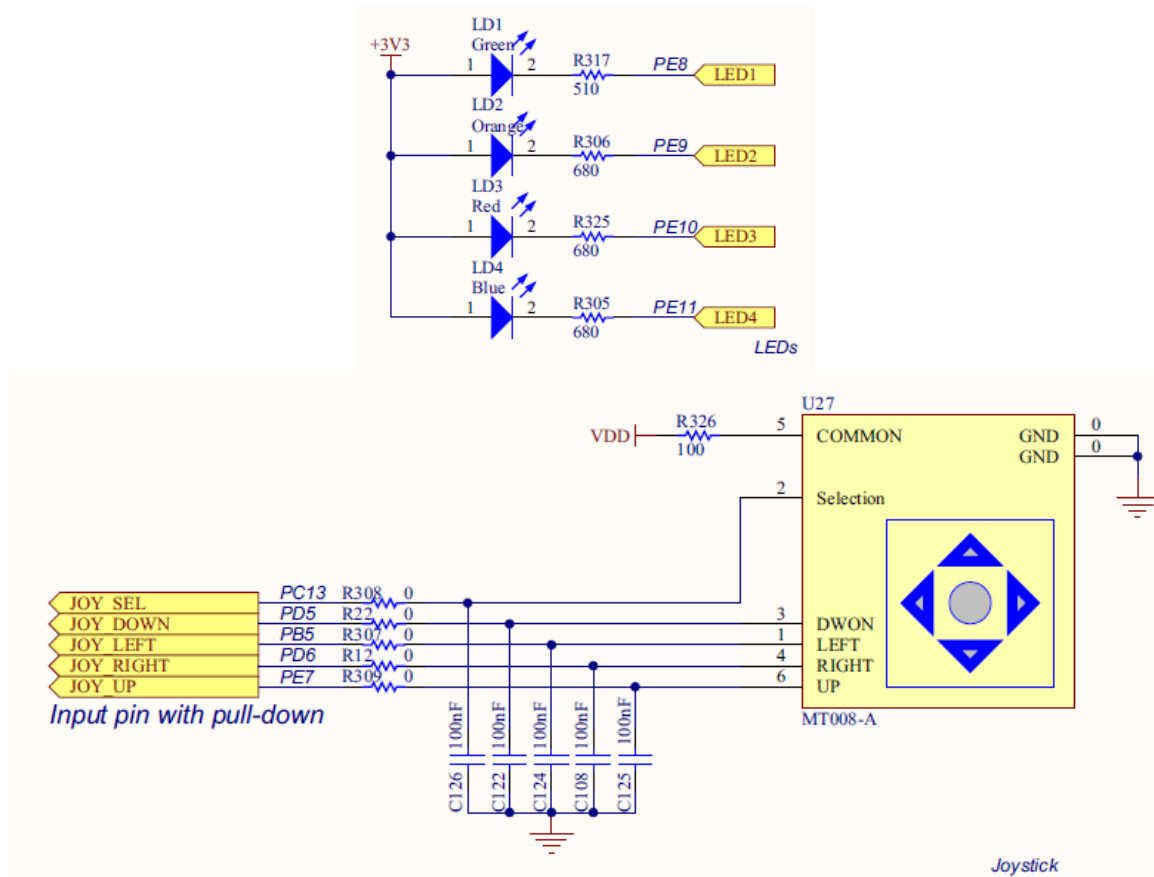
10: Pull-down

11: Reserved

Para esta práctica de laboratorio usted debe de construir un circuito que contenga lo siguiente:

- Joystick discreto equivalente a 5 interruptores momentáneos (pulsadores).
- 5 LEDs del color que usted prefiera con sus respectivos resistores para limitar la corriente a 15 mA.

A continuación, se presentan dos diagramas esquemáticos de ejemplo. Los nodos etiquetados como **PXn** corresponden al pin «n» del puerto paralelo «X» de un microcontrolador «Y». Su tarea consiste en seleccionar los pines y puertos disponibles en su tarjeta para diseñar la aplicación. En resumen, usted decide si habilita resistores internos de *pull-up* o *pull-down*, o bien si utiliza resistores externos; así como si prefiere encender los LEDs con un '1' o un '0' lógico.



Materiales y Equipo (Proporcionados por el Laboratorio)

- Tarjeta STM32F303 Evaluation Board.
- PC con Windows 10 y Keil μ Vision5.
- 1 Cable USB 2.0 A-Male a USB B-Male (Printer Cable).

Procedimiento

Siga los siguientes pasos prestando atención a las indicaciones de su instructor. Si tiene alguna duda o no está seguro de cómo proceder, pregunte a su instructor.

1. Asegúrese de conocer el hardware con el que trabajará. En particular verifique lo siguiente:
 - a. Mantenga la tarjeta alejada de piezas metálicas como joyas y llaves.
 - b. Mantenga la tarjeta alejada de piezas metálicas como joyas y llaves.

2. Cree un nuevo utilizando VS Code y Keil MDK6.
3. S programa (en C) debe encender los LEDs parcialmente cuando se cierren los interruptores (pulsadores) del Joystick, de acuerdo con la siguiente tabla:

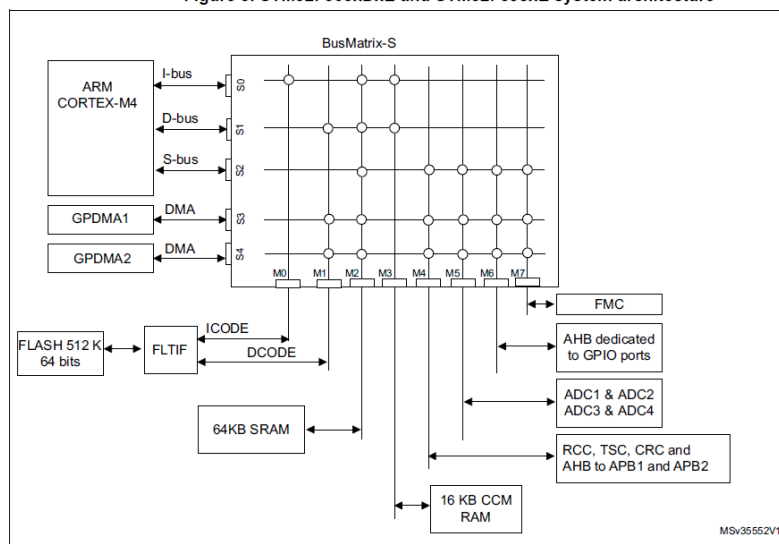
Joystick	Pin	Nombre del LED	Pin
Up	PE7	LD1	PE8
Right	PD6	LD2	PE9
Down	PD5	LD3	PE10
Left	PB5	LD4	PE11
Selection	PC13	LD7	PE13

Sí, cuando el usuario mueva el Joystick en el sentido indicado, debe iluminarse el LED indicado según la tabla. Si el Joystick está en reposo, el LED debe permanecer apagado.

4. Para lograr esto, es necesario configurar el módulo GPIO y los pines específicos que utilizará. Todo esto puede suceder al principio de su función principal. No se preocupe, los siguientes pasos le dirán qué hacer.
5. Suministre la señal de reloj a los módulos de GPIO necesarios (de acuerdo al diagrama esquemático que usted mismo diseño). Esto es necesario para habilitar los módulos mencionados. Si se intenta utilizar un módulo al que no se le ha suministrado una señal de reloj, ocurrirá un problema de hardware (Hardware Fault), lo cual debemos evitar.

Para suministrar la señal de reloj a los módulos de GPIO, conviene entender cómo funciona la arquitectura de bus del microcontrolador, llamada AMBA (Advanced Microcontroller Bus Architecture). A continuación, tenemos parte del diagrama de bloques:

Figure 3. STM32F303xDxE and STM32F398xE system architecture



El módulo GPIO está conectado al microcontrolador por medio de un bus llamado AHB (Advanced High-performance Bus) para el STM32F3 y AHB2 para el STM32G4. De hecho, GPIO y ADC son los únicos módulos que utilizan este bus de alta velocidad. El resto de los módulos utilizan un bus de menor velocidad llamado APB (Advanced Peripheral Bus).

Para proporcionar la señal de reloj a cualquier módulo, es necesario habilitar bits específicos en los registros **RCC->AHBENR** para el STM32F3 y **RCC->AHB2ENR** para el STM32G4. Cada módulo tiene un bit reservado en este registro para habilitar su señal de reloj. Puede leer más al respecto en el manual de usuario del microcontrolador, Capítulo Reset and Clock Control del Reference Manual.

Ya que es necesario escribir únicamente en bits específicos, usted debe prestar especial atención al código que escriba para implementar esta escritura. Dos cosas pueden salir mal:

- Podría no habilitar el módulo GPIO, en cuyo caso su programa dejará de funcionar correctamente al intentar utilizar los puertos paralelos.
 - Podría habilitar otros módulos por error. En este caso, podría modificar la funcionalidad esperada de su microcontrolador, y seguramente consumirá más energía que la necesaria por tener otro módulo funcionando.
6. Configure los pines del Joystick como entradas, y los pines del LED como salidas. Esto lo puede hacer escribiendo “1” a los bits que desea configurar como salidas, y “0” a los que desea configurar como entradas, en los registros de configuración nombrados con el siguiente formato **GPIOx->MODER**, donde **x** es el nombre del puerto. Por ejemplo, para volver salida el pin 7 del puerto A y volver entrada el pin 2 del puerto B, puede utilizar las siguientes líneas:

```
GPIOA->MODER |= MASK(2*7);  
GPIOB->MODER &= ~MASK(2*2);
```

No use esas líneas textualmente, ya que esos pines no son utilizados por la aplicación actual. Podría dañar la tarjeta Base Board o el microcontrolador si configura puertos sin cuidado. Recuerde que MODER necesita 2 bits para poder configurar la funcionalidad de cada pin.

Después de las dos líneas que incluyen las librerías necesarias que contienen todos los símbolos y estructuras definidas por CMSIS (Common Microcontroller Software Interface Standard), escriba lo siguiente: **#define MASK(x) (1<<x)**. En su reporte, explique qué hace esa función, qué tipo de función es, y qué diferencia hay entre utilizar esa función y lo que aparece al lado derecho de la asignación del paso anterior. Si no incluye estas tres explicaciones en su reporte, entenderemos que usted renuncia a obtener puntos por elaborar un reporte en esta práctica.

7. Es muy común que en los microcontroladores, los pines tengan múltiples funciones. Esto se hace para que el microcontrolador sea versátil, y para ahorrar pines, ya que rara vez una aplicación hace uso de todos los módulos de un microcontrolador. Normalmente, la función por default de los pines es la de puerto paralelo (GPIO). Por lo tanto, en esta aplicación no haremos uso de los **Alternate Function Registers**.
8. Para su aplicación, será necesario habilitar los pull-up o pull-down resistors (según su diseño) para cada uno de los pines configurados como entrada y que se encuentran conectados al Joystick de la tarjeta de evaluación.

9. Por ejemplo, para configurar el pull-down resistor del pin 2 del puerto B, puede escribirle al `pull-up/pull-down register` utilizando la siguiente línea:

```
GPIOB->PUPDR |= (2<<(2*2));
```

10. Ahora estamos listos para escribir el ciclo infinito. Para que su código sea legible, conviene utilizar símbolos significativos para el hardware utilizado. Utilice directivas de preprocesador para este fin, por *ejemplo*:

```
#define PuertoAIn      GPIOA->IDR
#define PuertoAOut     GPIOA->ODR
#define LEDcito        7
#define Switchecito    3
...
if(PuertoAIn & MASK(Switchecito)) //Leemos PA3
    PuertoAOut |= (MASK(LEDcito)); //Escribimos 1 en PA7
...
    PuertoAOut &= ~(MASK(LEDcito)); //Escribimos 0 en PA7
```

Es más: Puede implementar funciones para que su código sea aún más amigable. Por ejemplo:

```
Configurar_LEDs();
Configurar_Joystick();

while(1){
    if(arriba())
        encender(LED1);
    else if(abajo())
        encender(LED3);
    else if(derecha())
        encender(LED2);
    else if(izquierda())
        encender(LED4);
    else if(boton())
        encender(LED7);
    else
        apagar();
}
```

Esto reduce su tarea a implementar las constantes y funciones allí utilizadas.

11. Compile y descargue su programa.
12. Si su programa tiene algún problema, utilice sus habilidades de programador experimentado para depurarlo. Recuerde que siempre existe la posibilidad de que el problema provenga del autor del código. Entiéndase: Usted.
13. ¡Hemos terminado! Pídale a su auxiliar que verifique el funcionamiento de su aplicación.